

Probabilidad, MLE y MAP

Cómo un modelo aprende: maximizar la verosimilitud

Enrique Bolaños Abner Xocop

4 de agosto de 2026

Veritas · Libertas · Jvstitia

Agenda

1. ¿Por qué probabilidad?
2. Repaso express
3. Simular para entender
4. La verosimilitud
5. Máxima verosimilitud (MLE)
6. Máximo a posteriori (MAP)
7. Ejercicios
8. Cierre

Los datos son señal más ruido

Ninguna medición del mundo real es limpia:

- Encuestamos a 800 personas, no a 18 millones
- El sensor tiene error; el formulario tiene typos
- Dos clientes idénticos en los datos toman decisiones distintas

Modelar es separar lo que se repite (la **señal**) de lo que no (el **ruido**). La probabilidad es el idioma para hacerlo sin mentir: nos obliga a decir *qué tan seguros* estamos.

Sin incertidumbre no hay ciencia de datos, hay adivinanza

Variable aleatoria y las distribuciones que usaremos

Una **variable aleatoria** asigna un número a un resultado incierto. Su **distribución** dice qué valores toma y con qué probabilidad.

Distribución	Modela	Parámetros
Bernoulli	un sí/no (¿compró?)	p
Binomial	cuántos síes en n intentos	n, p
Normal	medidas continuas, errores	μ, σ^2
Poisson	conteos por unidad de tiempo	λ

Toda esta sesión se apoya en las dos primeras. La Normal reaparece el jueves con el Teorema Central del Límite, y Poisson solo la mencionamos para que sepan que existe.

Esperanza y varianza: centro y dispersión

La **esperanza** es el promedio a largo plazo:

$$E[X] = \sum_x x P(X = x)$$

La **varianza** mide qué tanto se aleja X de su centro:

$$\text{Var}(X) = E[(X - E[X])^2] = E[X^2] - E[X]^2$$

Para Bernoulli(p): $E[X] = p$ y $\text{Var}(X) = p(1 - p)$. Noten que la varianza es máxima en $p = 0,5$: la moneda justa es la más impredecible de todas.

Conjunta, condicional y la regla de Bayes

- **Conjunta:** $P(A, B)$, que pasen ambas cosas
- **Condicional:** $P(A | B) = \frac{P(A, B)}{P(B)}$, qué pasa *si ya sé* que ocurrió B
- **Independencia:** $P(A, B) = P(A) P(B)$

Despejando la conjunta de las dos formas posibles sale la **regla de Bayes**:

$$P(H | D) = \frac{P(D | H) P(H)}{P(D)}$$

Se lee: mi creencia sobre la hipótesis H después de ver los datos D es mi creencia previa, corregida por qué tan bien H explica lo que vi.

Bayes en acción: el filtro de spam

Un correo trae la palabra "oferta". ¿Es spam?

- $P(\text{spam}) = 0,20$ (prior: 1 de cada 5 correos)
- $P(\text{oferta} \mid \text{spam}) = 0,60$
- $P(\text{oferta} \mid \text{no spam}) = 0,05$

$$P(\text{spam} \mid \text{oferta}) = \frac{0,60 \times 0,20}{0,60 \times 0,20 + 0,05 \times 0,80} = \frac{0,12}{0,16} = 0,75$$

MÁS ALLÁ DEL CÓDIGO

Este cálculo exacto, repetido palabra por palabra, es todo el algoritmo de Naive Bayes que veremos el 13 de agosto. No hay magia: hay conteo y regla de Bayes.

NumPy como laboratorio de probabilidad

Cuando la fórmula no da intuición, simulen: es gratis y no se equivoca.

```
1 import numpy as np
2 rng = np.random.default_rng(42)
3 x = rng.binomial(1, 0.3, size=1000)  # moneda p=0.3
4 x.mean()                           # ~0.30
5 alturas = rng.normal(170, 10, size=1000)
6 alturas.mean(), alturas.std()      # ~170, ~10
7 # Ley de los grandes numeros: el promedio converge
8 n = np.arange(1, 1001)
9 promedio = np.cumsum(x) / n
10 promedio[9], promedio[99], promedio[999]
```

Con 10 tiros el promedio brinca; con 1000 se estaciona cerca de p . Esa convergencia es la razón por la que más datos ayudan.

De la probabilidad a la verosimilitud

Hasta ahora íbamos hacia adelante: *dado el parámetro, ¿qué datos espero?* En ciencia de datos vamos al revés.

Probabilidad	conozco θ , pregunto por los datos
Verosimilitud	tengo los datos, pregunto por θ

Es la **misma fórmula**, leída con la variable contraria fija:

$$L(\theta) = P(\text{datos} \mid \theta)$$

Ojo: $L(\theta)$ **no** es una distribución sobre θ . No integra a 1. Es un puntaje de plausibilidad.

La verosimilitud de la moneda

Lanzo 10 veces y salen 7 caras. ¿Qué p hace esos datos más plausibles?

```
1 k, n = 7, 10
2 p = np.linspace(0.01, 0.99, 99)
3 L = p**k * (1 - p)**(n - k)
4 p[np.argmax(L)]           # 0.70
```

Con $p = 0,1$ la verosimilitud es casi cero: una moneda tan sesgada casi nunca produce 7 caras. Con $p = 0,7$ es máxima. La curva tiene un solo pico y ese pico tiene nombre.

Elegir el pico de $L(\theta)$ se llama MLE

MLE de la moneda, paso a paso

Con k caras en n tiros independientes:

$$L(p) = p^k (1 - p)^{n-k}$$

$$\ell(p) = k \log p + (n - k) \log(1 - p)$$

$$\frac{d\ell}{dp} = \frac{k}{p} - \frac{n - k}{1 - p} = 0$$

$$k(1 - p) = (n - k)p \implies \hat{p} = \frac{k}{n}$$

El resultado es el que cualquiera habría adivinado (7 de 10 da 0.7), pero ahora sabemos *de dónde sale*: no es sentido común, es el máximo de una función que escribimos nosotros.

¿Por qué siempre el logaritmo?

- **Productos a sumas:** $\log \prod_i f(x_i) = \sum_i \log f(x_i)$, y derivar sumas es trivial
- **Estabilidad numérica:** multiplicar 10,000 probabilidades da 10^{-4000} , que en `float64` es exactamente 0
- **No cambia el argmax:** $\log$ es estrictamente creciente, así que el máximo está en el mismo lugar

Por convención los optimizadores *minimizan*, así que en la práctica minimizamos $-\ell(\theta)$, la **negative log-likelihood**.

MÁS ALLÁ DEL CÓDIGO

Cuando en scikit-learn o PyTorch vean una loss llamada log loss o cross entropy, es exactamente esto con otro nombre.

MLE para la Normal

Con $x_1, \dots, x_n$ independientes de una $\mathcal{N}(\mu, \sigma^2)$:

$$\ell(\mu, \sigma^2) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

Derivando respecto a μ , solo sobrevive la suma de cuadrados:

$$\hat{\mu} = \frac{1}{n} \sum_i x_i, \quad \hat{\sigma}^2 = \frac{1}{n} \sum_i (x_i - \hat{\mu})^2$$

Maximizar la verosimilitud de una Normal equivale a **minimizar la suma de errores al cuadrado**. Ese es el puente entre esta sesión y la regresión lineal del 18 de agosto.

La curva de log-verosimilitud en código

```
1 k, n = 7, 10
2 def log_lik(p):
3     return k*np.log(p) + (n-k)*np.log(1-p)
4 p = np.linspace(0.001, 0.999, 999)
5 ll = log_lik(p)
6 p[np.argmax(ll)]          # 0.700
7 # La Normal: el MLE coincide con las funciones de NumPy
8 d = rng.normal(170, 10, size=500)
9 mu_hat = d.mean()
10 sigma_hat = d.std()      # divide entre n, como el MLE
```

Buscar el máximo en una grilla es la versión bruta. Cuando θ tenga 50 dimensiones usaremos gradiente descendente, pero la idea es la misma: subir la colina de ℓ .

Agregar lo que ya sabíamos: el prior

Lanzo una moneda **2 veces** y salen 2 caras. El MLE dice $\hat{p} = 1$: la moneda nunca dará escudo. Nadie cree eso.

El problema es que el MLE ignora todo lo que sabíamos antes de ver los datos. Bayes nos deja incorporarlo:

$$\underbrace{P(\theta | D)}_{\text{posterior}} \propto \underbrace{P(D | \theta)}_{\text{verosimilitud}} \times \underbrace{P(\theta)}_{\text{prior}}$$

El **MAP** es el θ que maximiza el posterior, no la verosimilitud sola.

MLE responde: ¿qué explica mis datos? MAP: ¿y qué es razonable?

La moneda con prior Beta

Un prior $\text{Beta}(\alpha, \beta)$ sobre p se lee como "*antes de empezar, ya vi $\alpha - 1$ caras y $\beta - 1$ escudos imaginarios*". Con $\text{Beta}(2,2)$ (creo que la moneda es más o menos justa):

$$\hat{p}_{\text{MAP}} = \frac{k + \alpha - 1}{n + \alpha + \beta - 2} = \frac{k + 1}{n + 2}$$

Datos	k/n (MLE)	MAP Beta(2,2)	Diferencia
2 de 2	1.000	0.750	grande
20 de 20	1.000	0.955	menor
700 de 1000	0.700	0.700	casi nula

Con pocos datos el prior manda; con muchos, los datos lo aplastan.

MAP es MLE regularizado

Tomando logaritmo al posterior, el producto se parte en dos:

$$\log P(\theta \mid D) = \underbrace{\ell(\theta)}_{\text{ajusta datos}} + \underbrace{\log P(\theta)}_{\text{penaliza rarezas}} + c$$

Si el prior sobre los coeficientes de una regresión es $\mathcal{N}(0, \tau^2)$, ese segundo término es exactamente $-\lambda \sum_j \beta_j^2$: la penalización **ridge**.

Prior Normal en los coeficientes	regularización L2 (ridge)
Prior Laplace en los coeficientes	regularización L1 (lasso)

Spoiler del 18 de agosto: regularizar no es un truco de ingeniería, es poner un prior por escrito.

Ejercicio 1: log-verosimilitud de Bernoulli

Encuentren el MLE recorriendo una grilla de valores de p .

```
1 rng = np.random.default_rng(3)
2 datos = rng.binomial(1, 0.35, size=200)  # 0s y 1s
3 def log_lik_bernoulli(p, datos):
4     # ¿Que va aqui?
5     # Pista: k = datos.sum(), n = len(datos)
6     pass
7 grilla = np.linspace(0.001, 0.999, 999)
8 # valores = np.array([log_lik_bernoulli(p, datos)
9 #                       for p in grilla])
10 # print(grilla[np.argmax(valores)], datos.mean())
```

Ejercicio 2: MLE de la Normal

Deriven los estimadores con código y compárenlos con NumPy.

```
1 muestra = rng.normal(170, 10, size=800)
2 # a) mu_hat: el promedio, sin usar .mean()
3 mu_hat = ...
4 # b) sigma2_hat: promedio de (x - mu_hat)**2
5 sigma2_hat = ...
6 # Verificacion (descomenten al terminar):
7 # print(mu_hat, muestra.mean())
8 # print(np.sqrt(sigma2_hat), muestra.std())
```

Pregunta para pensar: `muestra.std(ddof=1)` divide entre $n - 1$ y no entre n . ¿Cuál de las dos es el MLE?

Ejercicio 3: el prior se diluye

Muestren que el MAP se acerca al MLE cuando crecen los datos.

```
1 def mle(k, n):  
2     return k / n  
3 def map_beta(k, n, a=2, b=2):  
4     # ¿Que va aqui? Formula en la lamina del prior Beta  
5     pass  
6 verdadero_p = 0.7  
7 for n in [5, 20, 100, 1000, 10000]:  
8     k = rng.binomial(n, verdadero_p)  
9     # print(n, mle(k, n), map_beta(k, n))
```

Resolvemos los tres en el notebook de la sesión; lo que falte queda de práctica para la casa.

Lo esencial de hoy

- La probabilidad modela el **ruido**; Bayes actualiza creencias
- $L(\theta)$ es la misma fórmula leída como función de θ
- **MLE**: elegir el θ que hace los datos más plausibles ($\hat{p} = k/n, \hat{\mu} = \bar{x}$)
- Usamos log por sumas, estabilidad y mismo argmax
- **MAP** = MLE + prior = MLE **regularizado** (ridge, lasso)
- Casi todo el curso será: elegir un modelo y maximizar su verosimilitud

Próxima clase (jueves 6): Inferencia Estadística y A/B Testing

Se asigna la **HDT 2** (entrega: jueves 13 de agosto, 23:59)